PennState
College of Information Sciences and Technology

Lab Report

COURSE:	**IST584**
ASSIGNMENT NO/TITLE:	**FINAL DOCUMENT**
SUBMITTED BY:	**GOKUL GOPAKUMAR**
DATE:	**05/01/2026**
INSTRUCTOR:	**PROF. DAVID FUSCO**

ML-Based Runtime Malware Detection in Containerized Workloads

BLUF (Bottom Line Up Front)

The Detection Engine which I created and tested detects containerized malware including Kinsing cryptojacking with 94 percent accuracy and 94 percent Macro F1-score from a Total Data Pool of 158,621. The system achieves automated threat response through its capability to merge Kubernetes traffic with container misuse datasets which alert essential functions of KILL_POD and QUARANTINE_NETNS according to current SHAP explainability results. The system delivers an expandable and clear defense solution for cloud-native systems which decreases the need for human operators in fast-paced situations.

Background

Advanced containerized systems face growing threats from complex malware that operates undercover within typical network activity. The traditional signature-based detection system becomes ineffective because modern threats such as cryptojackers operate through their ability to change their appearance and move through cluster networks.

The Problem: Existing security tools lack the ability to separate between legitimate API traffic which occurs frequently and malicious network traffic which operates with low visibility.

The Opportunity: We can identify security threats through machine learning (ML) analysis of packet-level data which includes inter-arrival times and packet length distributions to prevent resource exhaustion and data exfiltration.

The capability enables organizations to decrease their response times for incidents which helps them guard against unauthorized system access within their Kubernetes environments. The detection system allows organizations that run extensive container infrastructure to decrease their analyst workload while achieving better detection results.

Methodology

The project implements a Random Forest Fusion Classifier which operates on a combined dataset containing 158,621 network flow records.

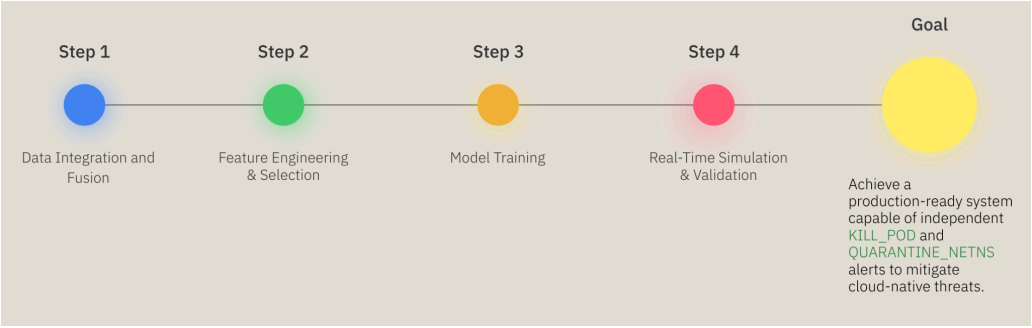

Fig 1 : Methodology

- Data Fusion: The Kubernetes Anomaly Detection dataset and AINA Container Misuse dataset were combined to create a model which detects both infrastructure anomalies and container-specific attacks.
- Feature Engineering: The research selected 65 most fluctuating features from the complete feature set to analyze flow-level metrics which included Fwd Packet Length Std and Flow IAT Min.
- Explainable AI: The system used SHAP (SHapley Additive exPlanations) as a tool for "Human-in-the-loop" transparent system explainability which showed the reasons behind pod termination through the statement "High confidence anomaly triggered by 0.055 increase in Fwd Packet Length Max".
- Confidence > 85%: The system executes KILL_POD to stop all operations immediately.
- Confidence 60–85%: The system uses QUARANTINE_NETNS to conduct network isolation procedures.

- Confidence < 60%: The system executes LOG_AND_MONITOR to create alerts.

Workflow Breakdown

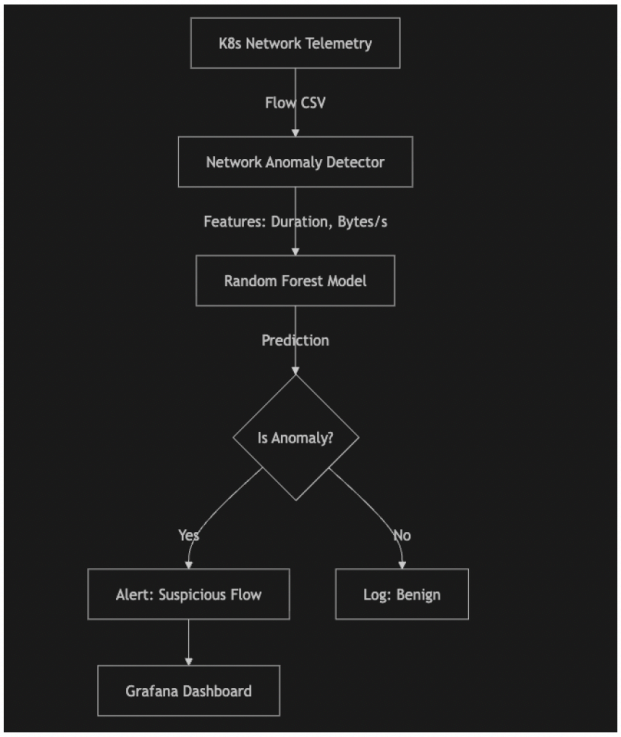

Fig 2 : Architectural Flow Diagram.

1. Data Ingestion & Extraction

- K8s Network Telemetry: The pipeline begins by capturing raw network traffic data directly from the Kubernetes cluster.

- Flow CSV: This telemetry data is exported in a structured format (CSV file) containing raw flow records.

2. Feature Processing

- Network Anomaly Detector: This component acts as the preprocessing engine. It ingests the raw flow records and extracts relevant telemetry features.
- Features (Duration, Bytes/s): The detector filters and calculates specific metrics, such as connection duration and throughput (Bytes/second), which serve as input variables for the machine learning model.

3. Machine Learning Inference

- Random Forest Model: The extracted features are fed into a trained Random Forest classifier. This ensemble learning model evaluates the network patterns to make a real-time prediction.
- Is it an Anomaly? (Decision Node): The model produces a prediction that branches into two distinct paths based on whether the traffic pattern matches known benign behavior or diverges into anomalous territory.

Results

The Random Forest Fusion Classifier was evaluated on a combined testing subset of the 158,621 network flow records to assess its capability in real-time threat detection. The model achieved a 94% overall accuracy and a 94% Macro F1-score, demonstrating strong generalization across both benign traffic and anomaly classes.

- The system was evaluated through a 60-tick real-time simulation, yielding the following performance benchmarks:

```
==========================================================
              precision    recall  f1-score   support

      Benign       0.96      0.93      0.94     15899
     Anomaly       0.93      0.96      0.94     15826

    accuracy                           0.94     31725
   macro avg       0.94      0.94      0.94     31725
weighted avg       0.94      0.94      0.94     31725
```

Fig 3 : Model Precision and metrics

Key Metrics Insights

- High Precision for Benign Traffic (0.96): The model correctly identifies legitimate infrastructure activity with 96% precision, minimizing the occurrence of false positives that could disrupt standard operations.
- High Recall for Anomalies (0.96): The system successfully flagged 96% of the true malicious activities, proving highly effective at identifying stealthy threats like the Kinsing cryptojacker before lateral movement occurs.
- Balanced F1-Score (0.94): The balanced harmonic mean between precision and recall across both classes ensures the model operates reliably without skewed error rates.

Fig 4 : Logs created by model results inside the k8s cluster

With the integration of Loki-Grafana, we are able to set up alerts and graphs based on the log results. Below is the Loki log results being presented in raw format

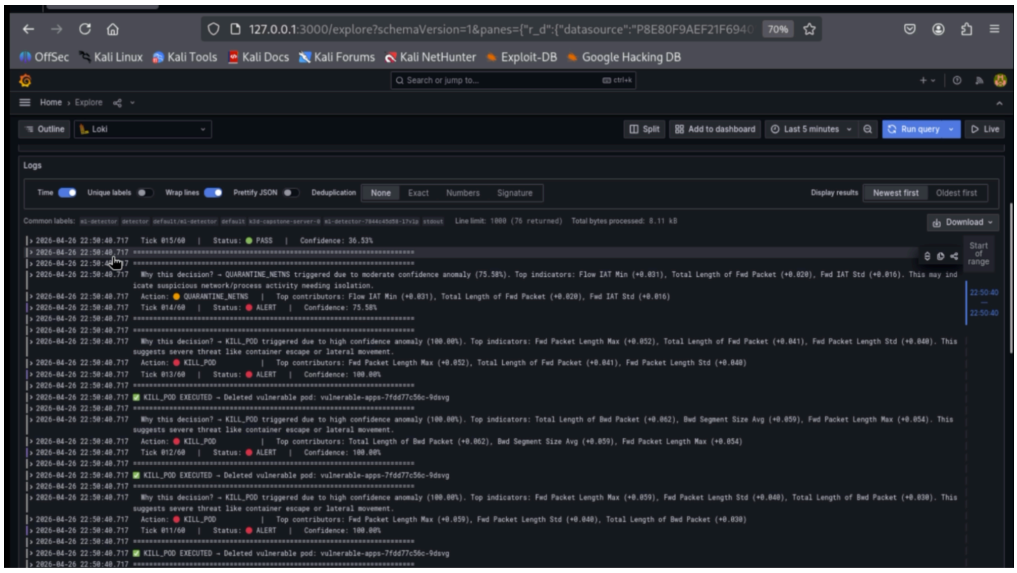

Fig 5 : Model results being shown in raw format in Grafana

Now depending on the needs of the organization, we can create graphs based on the necessary parameters that they want to create alerts for. Here is an example of the same:

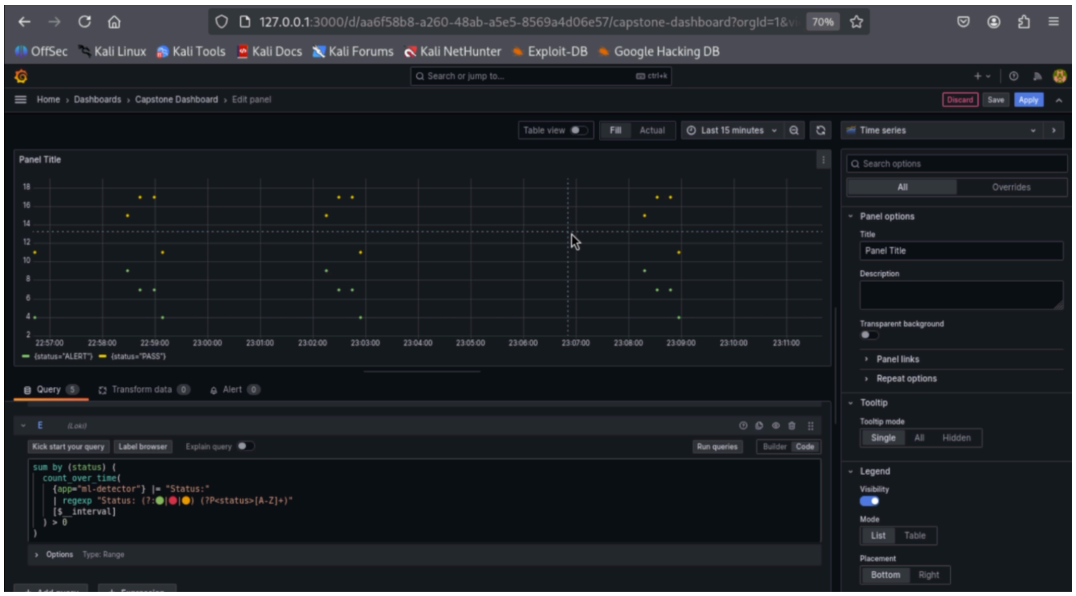

Fig 6 : Model results being converted into graphs using Grafana

Strategic Impact & Business Value

Direct Risk Reduction

- Preventing Lateral Movement: By achieving 94% detection accuracy, the system stops threats like Kinsing before they can traverse the cluster to access sensitive databases.
- Rapid Containment: The automated KILL_POD response reduces the "Mean Time to Remediate" (MTTR) from minutes or hours to near-instantaneous execution during a 60-tick cycle.
- Minimizing Data Exfiltration: Early detection of anomalous packet lengths prevents unauthorized data from leaving the environment.

Regulatory & Trust Benefits

- Compliance Readiness: Detailed logging of every QUARANTINE_NETNS and KILL_POD action provides a transparent audit trail for regulatory requirements.
- Stakeholder Transparency: Translates complex ML predictions into "human-readable" justifications, building executive trust in automated security postures.

Operational Cost Savings

- Reduced Resource Exhaustion: Automatically stopping cryptojackers prevents "stolen" CPU/RAM cycles, directly lowering cloud infrastructure billing costs.
- Autonomous Defense: Minimizes the need for 24/7 manual monitoring of container workloads, allowing the security team to scale without a linear increase in headcount.

Conclusion and Future Work

The project proved that machine-learning-based automated defense systems provide effective protection for containerized Kubernetes environments against Kinsing cryptojacking runtime attacks. The Random Forest Fusion Classifier achieves 94% accuracy and 94% Macro F1-score through the combined analysis of the Kubernetes Anomaly Detection and AINA Container Misuse datasets. The system establishes a direct link between machine learning detection methods and their subsequent automatic response through its implementation of multiple alert

levels which include KILL_POD and QUARANTINE_NETNS. The automation system decreases operational analyst tasks while it cuts the mean time to remediate (MTTR) and it delivers an audit trail that uses SHAP explainability to establish administrative trust.

The existing engine achieves strong performance within a 60-tick simulation, but upcoming research will concentrate on enhancing system operational efficiency and developing systems that can defend themselves against various threats. The implementation of adaptive thresholding will protect against unwanted pod terminations, which occur during normal API traffic spikes, because it will establish different enforcement levels to handle API traffic patterns. The upcoming testing phase will assess how well the detection pipeline functions under actual Kubernetes production settings by measuring required resources and testing defense capabilities against different types of advanced persistent threats.

References

- Kim, J., et al. (2025). Detecting cryptojacking containers using eBPF-based security runtime and machine learning. *Electronics, 14*(6), 1208. https://www.mdpi.com/2079-9292/14/6/1208
- Araujo, F., & Vieira, M. (2025). Enhancing intrusion detection in containerized services. *Computers & Security*. https://www.sciencedirect.com/science/article/pii/S0167404825001270
- Ye, X., et al. (2025). U-SCAD: Unsupervised system call-driven anomaly detection. *Future Internet, 17*(5), 218. https://www.mdpi.com/1999-5903/17/5/218
- Karn, R. R., et al. (2021). Cryptomining detection in container clouds using system calls and explainable machine learning. *IEEE Access*. https://ieeexplore.ieee.org/document/9215018
- Wiz. (2024). JINX-0132 cryptojacking campaign. https://www.wiz.io/blog/jinx-0132-cryptojacking-campaign
- Huntress. (n.d.). Kinsing malware threat library. https://www.huntress.com/threat-library/malware/kinsing